

Platevision

How the project works

Final-project explanation | Evidence snapshot: 4 October 2026

A detailed guide to the application, its self-trained character CNN, its adapted PaddleOCR reader, and the experiments behind the final configuration.

The important result: on 99 evaluated plate crops combined, the active PaddleOCR route read **77 complete plates correctly (77.8%)** and achieved a **95.1% character score**. The self-trained CNN route read **31 complete plates correctly (31.3%)** with a **72.5% character score**. These are development/validation results, not independently certified real-world accuracy.

What this report explains

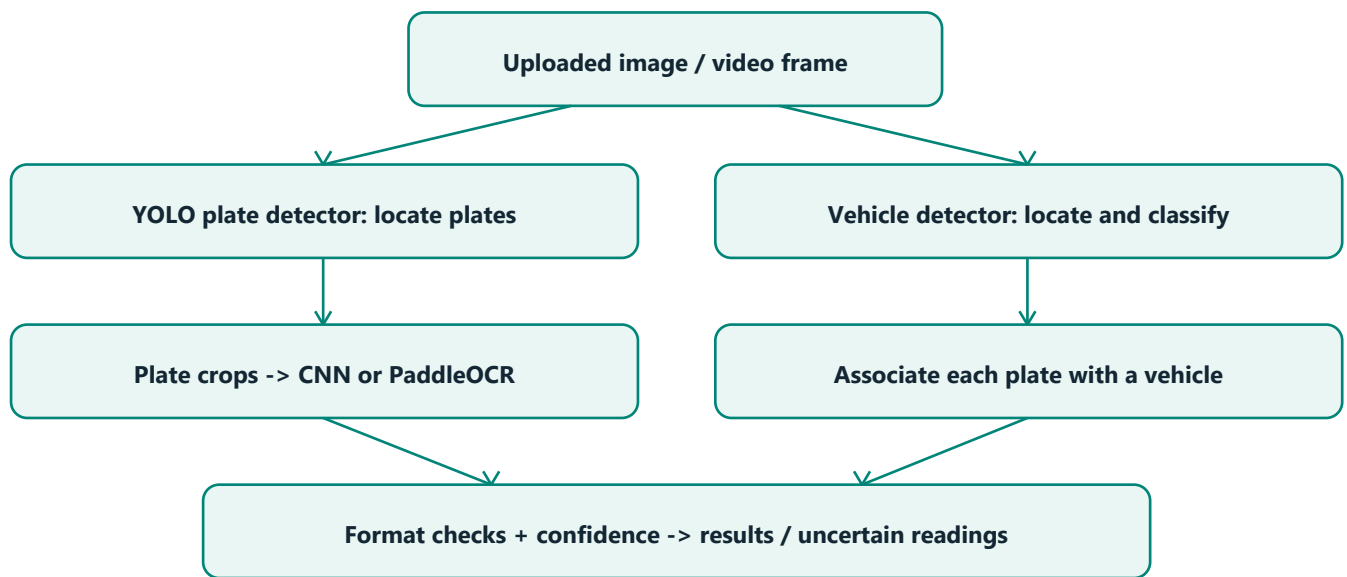
The project separates finding a plate from reading it. It also separates reading text from deciding whether a result is confident enough to show as accepted. Understanding those boundaries explains why a visible plate can be missed, a correctly read plate can be rejected, and a high character score can coexist with a lower complete-plate score.

Topic	Pages
Pipeline, components and detection	2-4
Your CNN and its training	5-6
PaddleOCR and local adaptation	7-8
Indian-format decoding and measured accuracy	9-11
Experiment history, datasets and failure analysis	12-14
Final configuration and evidence index	15-16

The report describes the saved final project, rather than promising a target accuracy. Historical experiments are identified separately from the active models so that improvements are not added together across incompatible tests.

Evidence: Local code and saved reports in outputs/PlateVision1; source keys are listed on page 16. No new training was performed to create this report.

From a photograph to a result



1. Locate objects and crop each plate

The app accepts an image or processes sampled video frames. A plate detector searches the image for number plates. A separate vehicle detector supplies boxes and categories such as car, motorcycle, bus or truck. Vehicle detection is not a prerequisite for plate detection: a plate can be found even when its vehicle is unknown.

2. Read the crop through one of two routes

The main Recognition page uses the strongest selected route: the external YOLO plate detector followed by adapted PaddleOCR. The Compare page also runs your own character CNN on plate crops. Both routes share plate localization, making the reader comparison more meaningful, although their preprocessing differs.

3. Decode, filter and associate

The reader proposes text and confidence values. Indian registration patterns help choose between plausible character alternatives. A confidence gate and structural format check decide whether to accept the reading. Geometry links each plate to a detected vehicle. A car/truck mistake can therefore occur even when its plate text is correct.

4. Present the result

Accepted unique plate strings receive session serial numbers, with vehicle type and text in the results table and a CSV export. Uncertain readings remain inspectable. The comparison view sorts candidates by decreasing reading confidence and keeps accepted/uncertain status separate. A unique-plate count is not a guaranteed count of every vehicle in the scene.

Evidence: [S1] app.py; src/pipeline.py; src/comparison.py; src/comparison_ui.py; src/association.py.

Why there are two readers

Component	Origin and role
Your character CNN	Project-defined architecture, trained from random initialization and later updated using labelled character data. Reads separately segmented glyphs.
PaddleOCR reader	External pretrained text models, integrated locally and adapted by training the final CTC projection. Reads complete text lines.
Active plate detector	External pretrained YOLOv9-t ONNX model. Finds plate boxes; it does not read the text.
Vehicle detector	Torchvision pretrained SSDLite320 with MobileNetV3-Large. Supplies vehicle boxes and category names.
Your experimental plate detector	Project-defined anchor-free detector, trained on synthetic and real boxes. Retained for experiments; not the main detector.

The reason for the comparison

Your CNN demonstrates how a recognition model can be built and trained locally. PaddleOCR provides a stronger practical reader that benefits from extensive prior text training. The comparison shows the cost of character segmentation, limited training diversity and smaller local training runs. It is a comparison between two approaches, not a claim that the application contains only two neural networks.

What is genuinely your work?

The CNN architecture and training pipeline are project code, with learned weights originating from your own training. Downloading labelled images does not turn it into a pretrained model. The OCR route reuses pretrained weights, but the integration, plate handling, local head adaptation, format decoder, evaluation and user interface are project work. It would be inaccurate to claim that PaddleOCR itself was invented or trained from scratch here.

Older options still present in the source

The repository also retains a custom whole-plate CNN + bidirectional GRU + CTC pilot, a FastPlateOCR CCT-S v2 alternative, and classical OpenCV plate proposals. They explain earlier extra model options. They are not the two active comparison routes and their historical results should not be mixed into the current accuracy figures.

Evidence: [S1-S3, S13] Runtime choices, architectures and REAL_READER.md. Upstream attribution: [E1-E4].

How plate and vehicle detection work

The active YOLO plate detector

The detector receives a colour image, resized with padding to a 384 x 384 square and scaled to floating-point values. It predicts candidate plate boxes and scores. Candidates below 0.50 are removed. Overlapping duplicates are suppressed using a 0.40 intersection-over-union (IoU) threshold, with a maximum of 30 candidates.

For images whose longest edge exceeds 640 pixels, the code also searches four overlapping tiles, each covering 60% of the image dimensions. This gives small plates more input pixels than a single heavily reduced full image. It does not recover detail missing from the original photograph, and it costs extra inference time.

Your own plate detector

The retained custom detector uses grayscale 320 x 320 input and an FCOS-style, anchor-free design. Five downsampling stages feed a feature pyramid at strides 8, 16 and 32. Separate heads predict plate presence, four distances to box edges, and centerness. Training combines focal classification loss, generalized-IoU box loss and centerness loss. This is different from the character CNN.

Detector, same 61-image development set	Precision	Recall	AP50
Original custom detector	14.3%	15.2%	6.3%
Indian-focused custom training	46.5%	30.3%	24.1%
Full mixed-data custom training	20.0%	13.6%	6.2%
External YOLO, selected	89.9%	93.9%	93.1%

The set contains 66 reference plates. Precision and recall use score 0.50 and IoU 0.50; AP50 summarizes a precision-recall curve. These are box-detection metrics, not text accuracy. Labels are not fully reviewed and this is a development set. The Indian-focused custom model improved substantially, but still missed too many plates for the main application.

Vehicle category and association

SSDLite uses a MobileNetV3 feature extractor and lightweight detection heads. Pretrained COCO classes are mapped to car, motorcycle, bus and truck. A plate is usually assigned to the smallest vehicle box covering at least 80% of its area, with a small centre-distance fallback. Otherwise its type is Unknown. Vehicle-type accuracy has not been independently measured.

Evidence: [S2, S10] src/pretrained.py; src/plate_detector.py; src/vehicle_detector.py; detector evaluation reports. [E3-E4].

Your CNN: from pixels to characters

First, cut the plate into individual glyphs

The plate becomes grayscale and is resized to 128 pixels high. Otsu and adaptive thresholding are each tried in two polarities, producing four candidate black-and-white masks. Connected components are filtered by size, shape, fill and edge position. A tilt-aware grouping procedure orders rows from top to bottom and characters from left to right.

Each retained component is resized without stretching its proportions. Its longest dimension is 24 pixels and it is centred on a 32 x 32 black canvas. The foreground character is white. This is the route that genuinely uses black-and-white character images for classification.

Layer	Output shape / function
Input	1 x 32 x 32 grayscale glyph
3 x 3 convolution, 32 channels + BatchNorm + SiLU; max pool	32 x 16 x 16
3 x 3 convolution, 64 channels + BatchNorm + SiLU; max pool	64 x 8 x 8
3 x 3 convolution, 128 channels + BatchNorm + SiLU; max pool	128 x 4 x 4
Flatten and fully connected + ReLU	2,048 features -> 256 features
Dropout, then output layer	Dropout 0.25; 256 -> 36 class scores
Softmax at inference	Probabilities for digits 0-9 and letters A-Z

Why this architecture?

Convolutions learn local strokes and combinations of strokes. Pooling reduces spatial size and sensitivity to small shifts. Batch normalization stabilizes feature scales, while dropout discourages reliance on a small number of features. A small network is understandable, trainable on a CPU and appropriate for a college demonstration. More layers alone would not fix bad crops or labels.

How a plate prediction is selected

The classifier scores glyphs from the candidate segmentations. The code considers format plausibility, confidence eligibility and character log-probabilities when choosing a reading. Its reported reading confidence is the lowest selected character probability. One weak glyph can therefore make the complete result uncertain even when most characters are clear.

Evidence: [S2-S3] architectures.py; character_segmenter.py; character_classifier.py; preprocessing.py.

How the CNN was trained and reviewed

Supervised learning, from scratch

For a labelled glyph, the model predicts 36 class scores. Cross-entropy loss measures how poorly those scores support the correct label; AdamW updates the weights to reduce that loss. An epoch is a pass through the training examples. A lower training loss shows better fit to those examples, but does not prove better performance on new photographs.

Synthetic characters provided an initial practice source. Real character crops, printed-character data and sampled EMNIST examples were later tried. Training used bounded CPU resources, mild shift/rotation/scale augmentation, class/source weighting and replay of earlier examples. Some conservative trials froze BatchNorm statistics to avoid destabilizing an already useful model.

The human review work mattered

The first review round produced 99 usable characters from 100 checks. A later round brought the total to 299 usable characters from 300 checks, with one exclusion. Corrections improve supervision because a clean image with the wrong label teaches the wrong mapping. However, the verified set still had no I or V examples and only one each of O, X and Z in the final review record.

Why more reviewed characters did not guarantee a gain

The reader has two error sources: cutting the image and classifying the cuts. A perfect classifier cannot label a missing cut, and a merged pair of letters is not a valid single-character input. In one review, 20 of 32 wrong plate predictions had a different character count from the reference. That points to segmentation or sequence-length problems, although it does not prove classification is otherwise correct.

The checkpoint actually retained

The active character checkpoint is the conservative model at **models/character_conservative_v1/last.pt**. Its selected blend kept 75% of the preceding model and 25% of a conservative update. A later large-data run reached 16/45 exact plates instead of 14/45, but increased character edit errors from 118 to 140 and accepted fewer correct readings. It was not promoted because the trade-off was mixed.

Do not report 100% real-character accuracy. The saved 216/216 synthetic-glyph result is a synthetic validation result. The 299 reviewed real glyphs were training material, not an independent accuracy test.

Evidence: [S6-S8] Review, conservative-update, segmentation and combined-training records.

PaddleOCR: reading whole text lines

Two pretrained text components

The active OCR route uses RapidOCR ONNX exports of **ch_PP-OCRv5_det_mobile** for text detection and **en_PP-OCRv5_rec_mobile** for English recognition. These are the recorded model identities in this project. The application runs them locally with ONNX Runtime; it does not send each plate to a cloud OCR API.

Find and straighten the text

Within a plate crop, the text detector produces a probability map. Thresholded regions become candidate text lines, and a rotated rectangle is used to straighten each line with a perspective transform. Lines are ordered using their position and tilt. Up to eight lines can be considered, with a whole-plate fallback when no line is found. An isolated top line reading IND is ignored as a country marker.

Recognize a sequence, without cutting every character

Each line is scaled to 48 pixels high, retaining its proportions, and padded to a width between 320 and 1,280 pixels. The recognizer processes colour input and emits token probabilities along the line. Its final CTC projection maps sequence features to character/blank scores. CTC decoding removes blanks and collapses adjacent repeats: repeated symbols separated by a blank can still produce legitimate repeated characters.

The implementation retains a small set of alternatives at emitted positions and combines line candidates before format-aware selection. This is a bounded alternative search around greedy CTC output, not an unrestricted full CTC beam search. Therefore it can fix some substitutions but cannot freely reconstruct every missing or merged character.

Why this method was selected

Whole-line recognition avoids the CNN route's fragile requirement that every character be cut perfectly first. Pretrained text features also cover more fonts and imaging conditions than the small locally verified set. The mobile exports allow CPU inference. Plate localization, line ordering, glare, blur and unusual fonts still limit the result.

The black-and-white image in the OCR inspection panel explains preprocessing visually. It is not evidence that PaddleOCR reads the same individual binary glyphs as your CNN. PaddleOCR reads colour line/crop inputs.

Evidence: [S4] `src/whole_line_reader.py`; `src/paddle_candidate.py`; local model provenance. [E1-E2].

What we trained inside PaddleOCR

A local adaptation, not a complete rebuild

The pretrained feature extractor remained frozen. The project extracted its sequence features and trained only the final CTC projection matrix and bias using labelled text lines. The code first checked that the extracted head reproduced the original ONNX probabilities. The selected update was blended 50/50 with the preceding projection to make the change conservative.

Stage	Training material	Measured outcome
Initial projection update	76 screened, human-labelled text lines	26 development lines: 15 -> 16 exact. On 45 plates at 0.95: correct accepted 25 -> 27; raw exact 31 -> 31.
Final combined projection update	981 lines from 838 plate crops; 102 reviewed line labels and 879 source labels	26 development lines: 16 -> 18 exact. On 45 plates: raw exact 34 -> 34; character errors 21 -> 18.

What the final training run did

The final run used CTC loss and AdamW at learning rate 0.0001 for up to eight epochs. The selected epoch was the first. Candidate crops with unsuitable layout were quarantined before line training. Since a plate can yield more than one line, line counts and plate counts are different quantities.

How the final model was chosen

On the 45-crop development set, the new weights at threshold 0.90 accepted 33 correct and 4 incorrect readings. Raising the selected gate to 0.91 gave 31 correct and 3 incorrect accepted readings. On the additional 54 crops, raw exact plates increased from 42 to 43 and edit errors fell from 31 to 29. However, accepted results changed from 35 correct/1 wrong for the previous configuration to 34 correct/2 wrong for the new one.

The final model was selected for improved underlying reading, not because every confidence-filtered metric improved. The active folder is **models/paddle_combined_v2**. Describing this as full-network PaddleOCR fine-tuning would be incorrect; it is final-layer adaptation followed by integration and decoding improvements.

A cautious retry

For some small, uncertain crops, the reader retries at twice the image size. A higher-confidence retry is used only if it predicts exactly the same string. This can stabilize a reading but cannot turn invented detail into real information. Its internal 0.95 retry trigger is different from the final 0.91 acceptance threshold.

Evidence: [S4, S9] PADDLE_PROJECTION_UPDATE.md; adaptation.json; training_report.json; development_evaluation.json; combined_new_validation.json.

Indian-format decoding: what improved

Use a prior, while preserving image evidence

A common registration such as **MH12AB3456** separates into state letters MH, district digits 12, series letters AB and serial digits 3456. Position helps distinguish similar shapes such as S/5, O/0 and I/1. The project also handles a BH-style pattern and several older/shorter forms. These are software heuristics, not a registration database or proof that a number was legally issued.

The decoder checks known state prefixes and expected letter/digit positions. It permits at most two substitutions, using model-supported alternatives with probability at least 0.02 and at least 5% of the top choice. After the state prefix, substitutions are limited to confusion groups such as O/O/Q, 1/I/L, 2/Z, 5/S, 6/G and 8/B. It does not insert missing characters, delete extras or look up the expected answer.

Same 45 crops; weights unchanged	Before	After
CNN exact full plates	13/45 (28.9%)	14/45 (31.1%)
CNN character score / edit errors	70.2% / 131	73.2% / 118
OCR exact full plates	31/45 (68.9%)	34/45 (75.6%)
OCR character score / edit errors	94.3% / 25	95.2% / 21
OCR accepted correct / wrong, gate 0.90	29 / 7	29 / 3

Recorded examples include the CNN changing **6J15CD0564** to **GJ15CD0564**, and OCR changing **KH01CY6333** to **MH01CY6333**. No previously exact plate became wrong in this paired experiment. Some already-wrong CNN readings still became worse. The figures above predate the final PaddleOCR projection update on page 8.

Why format rules cannot solve everything

Two competing strings can both fit an allowed pattern. The user-confirmed DL35CW6192 and the alternative DL3SCW6192 illustrate why format alone cannot settle every ambiguity. The image and model probabilities must decide. The final acceptance check is also weaker than the decoder's known-state check, so passing the acceptance regex does not guarantee a valid state prefix or a real registration.

Evidence: [S5] FORMAT_AWARE_UPDATE.md; format_aware_comparison.json; src/format_decoder.py; src/validator.py.

Current plate and character results

Set and reader	Exact plates	Character score	Edit errors
45 development crops: CNN	14/45 = 31.1%	73.2%	118 / 440
45 development crops: PaddleOCR	34/45 = 75.6%	95.9%	18 / 440
54 additional crops: CNN	17/54 = 31.5%	72.0%	145 / 517
54 additional crops: PaddleOCR	43/54 = 79.6%	94.4%	29 / 517
Pooled 99 crops: CNN	31/99 = 31.3%	72.5%	263 / 957
Pooled 99 crops: PaddleOCR	77/99 = 77.8%	95.1%	47 / 957

What each number means

Complete-plate accuracy is the number of predicted strings exactly matching the reference divided by all evaluated plates. One wrong, missing or extra character makes that plate incorrect. These results count raw readings, including those marked uncertain by the confidence gate.

Character score is 1 minus character error rate (CER). CER is the total substitutions + deletions + insertions needed to transform predictions into reference text, divided by the total reference characters. Thus PaddleOCR's pooled score is $1 - 47/957 = 95.1\%$. It is a sequence edit-distance score, not a separately measured accuracy on isolated real glyphs.

Example, not project data: with ten 10-character plates, one error on each of five plates gives a 95% character score but only 50% exact plates. This is why 95% character performance does not imply 95% complete-plate performance.

How strongly can we trust these estimates?

The 45-crop set was repeatedly used for development and model/threshold decisions. The 54 additional crops were kept out of the final gradient updates and further threshold search, but their source labels were not fully manually verified and capture/near-duplicate provenance is incomplete. The reserved test split was not used in the final round. Pooling the sets is a descriptive summary, not a new independent test.

These are plate-crop reader results. They do not establish full-image end-to-end accuracy, vehicle-category accuracy, or performance on every Indian plate format. Upstream pretrained models may also have unknown overlap with public images. No 95% full-plate claim is supported by these reports.

Evidence: [S11] character_metrics.json; combined_new_validation.json. Pooled figures calculated from saved counts, not averaged percentages.

Confidence thresholds are not accuracy

What the final gates do

The active CNN uses **0.70** with the minimum chosen glyph probability. PaddleOCR uses **0.91** with the mean emitted-token score. Both also apply structural checks. These scores are not calibrated probabilities that the complete plate is correct, and the two score definitions are not directly comparable.

Current set / reader	Correct accepted	Wrong accepted	Uncertain
45 crops / CNN	9	2	34
45 crops / PaddleOCR	31	3	11
54 crops / CNN	11	5	38
54 crops / PaddleOCR	34	2	18
Pooled / CNN	20	7	72
Pooled / PaddleOCR	65	5	29

On the pooled crops, accepted-result precision is $20/27 = \mathbf{74.1\%}$ for CNN and $65/70 = \mathbf{92.9\%}$ for PaddleOCR. Coverage is $27/99 = \mathbf{27.3\%}$ and $70/99 = \mathbf{70.7\%}$, respectively. High accepted precision can coexist with many uncertain or missed results.

The recorded 0.95 versus 0.90 experiment

Before format-aware decoding, lowering the OCR gate from 0.95 to 0.90 left raw exact accuracy unchanged at 31/45. Correct accepted readings rose from 27 to 29, but wrong accepted readings rose from 2 to 7. Accepted precision fell from 93.1% to 80.6%. The lower gate displayed more results; it did not make the reader more accurate.

The CNN threshold comparison

At 0.65 and 0.70 the saved comparison accepted 9 correct and 2 wrong readings. At 0.75 it accepted 8 correct and 1 wrong; at 0.80 it accepted 7 correct and 1 wrong. All had 14/45 raw exact plates and 118 edit errors. The chosen 0.70 gate tied the best measured accepted F1 in that sweep and remained near the preceding 0.75 setting.

Increasing a gate usually removes uncertain results, including some correct ones. Decreasing it usually increases coverage and the risk of incorrect accepted output. Model training, better crops and better decoding change the readings themselves; threshold changes principally change which readings are accepted.

Evidence: [S12, S11] OCR_THRESHOLD_090.md; CNN_THRESHOLD_070.md; threshold comparison reports; current validation counts.

The improvement record

Each row below is a local before/after experiment. Different rows may use different checkpoints, thresholds or subsets; their gains must not be added together as if they were one controlled experiment.

Change	Observed effect	Decision / interpretation
Tilt-aware character ordering	Two crop subsets together: 10/45 -> 12/45 exact, same weights and thresholds.	Kept. Better ordering can help without retraining.
First 100 character reviews	99 usable glyphs; 12/45 -> 13/45 exact.	Kept. Correct labels and relevant crops helped.
Conservative CNN update	13/45 unchanged; edit errors 134 -> 131; wrong accepted 2 -> 1.	Kept conservative blend; improvement was character/filter quality.
300-review CNN retraining	Trials produced 11-12/45, versus 13/45 baseline.	Not promoted. More labels alone did not overcome segmentation limits.
Expanded segmentation and deskew trials	Several reached 14/45, but wrong accepted often rose from 1 to 2 or 4.	Not promoted as a general improvement. Edge-preservation trial fell to 12/45.
Indian-format alternative decoding	CNN 13 -> 14/45; OCR 31 -> 34/45.	Kept. Clear paired gains; see page 9.
Large character-data training	Best later run 16/45, but 140 edits versus active 118; additional set also mixed.	Not promoted. More exact plates but worse overall character errors.
Indian-focused plate detector training	Box recall 15.2% -> 30.3%; AP50 6.3% -> 24.1%.	Useful custom-model improvement; external detector still stronger.
Final PaddleOCR head adaptation	45 crops: 21 -> 18 edits, 34 exact unchanged. Extra 54: 42 -> 43 exact, 31 -> 29 edits.	Promoted with gate 0.91; accepted precision did not improve on every set.

Other learning-rate, normalization, mild-augmentation and BatchNorm-freezing trials did not produce a uniformly better CNN. Some lower training losses did not transfer to real plates. This is useful experimental evidence, not proof that no future method could improve the project.

Evidence: [S5-S10, S12] Saved experiment documents and evaluation JSON; no results inferred from screenshots alone.

Datasets, cleaning and your reviews

Source group	How it was used / excluded
Plate-box archives 2 and 5	Duplicate collections: 2,083 images and 2,021 YOLO label files. Screened material used where suitable; box labels do not provide text automatically.
Archives 3 and 9	Duplicate 47-photo, 52-box collection. Not counted twice as new data.
Archive 4	1,690-image OCR source collection. Screened text/crop material contributed to reader preparation.
Archive 6	1,698 images, 1,697 XML annotations; 387 screened candidate crops. 361 format-usable records split 251 train / 54 validation / 56 test; 26 questionable labels excluded.
Archive 7	35,500 printed glyphs covering 35 classes; no O class. Used in character-training trials.
Archive 8	EMNIST sample of 1,440 examples, 40 per class, from official training data. Not the entire archive.
Archive 10	Pakistan vehicle/plate-box data: 3,913 training images. Used for detection; Pakistan text labels were not used as Indian OCR targets.
3113449.zip	Unavailable in the recorded training round; not silently counted as trained data.

What “using all the datasets” meant

Useful sources were matched to their task, deduplicated or screened, and tried in the appropriate training run. The mixed detector set contained 4,238 images: 325 Indian plus 3,913 Pakistan training images. The large CNN manifest contained 43,998 rows across verified, synthetic, printed, EMNIST and weakly aligned glyphs. The final OCR adaptation used 981 text lines. These counts are not interchangeable.

What manual verification contributed

The record includes 11 dataset-image reviews, 200 OCR text reviews and 300 character checks, of which 299 were usable. Reviewing means checking the crop and true label; it does not itself launch training. Weakly aligned glyphs used matching character counts and model-agreement screening, so they remain weaker evidence than human-verified labels.

Exact hashes and registration grouping reduce obvious leakage, but cannot guarantee that adjacent captures, edited copies or pretrained sources never overlap. Public dataset links belong in the GitHub README; dataset licensing and provenance remain separate from whether model weights were trained from scratch.

Evidence: [S8, S13] COMBINED_DATASET_TRAINING.md; DATASETS.md; NEW_DATASET_TRIAL.md; SECOND_DATASET_TRIAL.md; review records.

Where errors enter the pipeline

Symptom	Likely stage	How to inspect it
A visible plate has no box	Plate localization or filtering	Inspect detector proposals, small-plate resolution and rejected scores before blaming OCR.
Box exists but no accepted result	Reading, format or confidence gate	Read the uncertain text and score. A correct string can still fall below the gate.
Missing or extra CNN characters	Segmentation or line ordering	Inspect binary masks and character crops. Compare the number of components with the reference.
S/5, O/0, B/8 substitution	Character/sequence prediction	Check image detail and alternatives. Format can help only when it distinguishes plausible choices.
Car shown as truck	Vehicle classification or association	Inspect the vehicle detector result separately. Plate-text training will not fix its category.
Grille, sign or other pattern treated as a plate	False-positive localization	Check negative examples, detector confidence and box quality; do not accept arbitrary text because it looks structured.

Examples from the project feedback

The scooter example produced raw KL02BP7403 but was initially rejected at a stricter gate despite an 88.1% OCR token score. That is different from never finding the plate. Tilted motorcycle plates and multi-vehicle scenes exposed localization and line-reading weaknesses. The SUV labelled Truck exposed a vehicle-category error independent of the recognized registration.

Why four plates in one image are harder than one crop

Each plate must first survive localization and cropping, then be read and pass acceptance. A missed box cannot be recovered by an excellent reader downstream. Small distant plates, decorative fonts, two-row layouts, collage boundaries and occlusion add different failure modes. Measuring only supplied plate crops removes localization from the test and therefore cannot describe the entire application.

What remains possible, without overstating this version

Better independent labels, more representative hard negatives, stronger plate localization and further sequence-model adaptation could help. The custom whole-plate CTC pilot already exists; treating it as an entirely untried idea would be misleading. The final project is a documented learning system with measured limits, not evidence that every possible improvement has been exhausted.

Evidence: [S1, S6-S8, S13] Inspection UI, segmentation trials and retained reader pilot; user-supplied failure examples.

Final configuration and how to explain it

Setting	Final selected behaviour
Main Recognition	YOLOv9-t plate boxes + adapted PaddleOCR, gate 0.91
Compare models	PaddleOCR and own character CNN; candidates sorted by confidence; acceptance shown separately
Own character reader	Conservative CNN checkpoint; gate 0.70
Vehicle information	Pretrained SSDLite320 MobileNetV3-Large; score gate 0.50
Image handling	Main full-crop rectification off; Paddle text-line perspective correction remains active
Runtime	Local CPU inference; two CPU threads; ONNX sequential execution
Limits	100 MB upload limit; 24-megapixel image limit; video processing capped at 30 seconds with a target sampling rate of 2 frames/second

The sampling rate is a processing policy, not a benchmarked real-time speed guarantee. CPU-only operation and bounded threading suit the Intel Core Ultra 7 laptop used for this project. Installing or downloading dependencies initially is separate from local inference after setup.

A clear project explanation

"I built a license-plate recognition application with two readers. My own CNN is trained from scratch to classify segmented characters. The stronger reader uses pretrained PaddleOCR, with a locally trained final CTC layer and Indian-format decoding. The system also integrates plate and vehicle detectors. I compare complete-plate accuracy, character edit score and accepted-result precision separately, and document where the methods fail."

How to reproduce the reported comparison

Keep the recorded checkpoints, crop splits and labels fixed. Evaluate raw text against reference text before applying acceptance gates, count character edit errors, then report accepted correct, accepted wrong and uncertain counts separately. For a full-image test, additionally annotate every target plate box and vehicle category. Do not tune on that final test after seeing its results.

Terms used in this report

Checkpoint: saved model weights. **Epoch:** one training-data pass. **Loss:** the optimization penalty, not an accuracy percentage. **IoU:** box overlap divided by box union. **CTC:** a sequence-learning/decoding method that aligns text without character-boundary labels. **Validation:** data used to assess/select models. **Test:** an untouched final evaluation set.

Evidence: [S1, S9, S11-S12] config.py, pipeline defaults and final evaluation records. The project was not modified or retrained for this report.

Evidence and references

Local paths below are relative to **outputs/PlateVision1**, the inspected project evidence folder. This index makes the claims traceable without embedding datasets in the report. The final runnable and GitHub packages may reorganize documentation; retain these reports when reproducing the results.

- [S1] app.py; config.py; src/pipeline.py; src/comparison.py; src/comparison_ui.py; src/association.py.
- [S2] src/architectures.py; src/plate_detector.py; src/pretrained.py; src/vehicle_detector.py; training/losses.py.
- [S3] src/character_classifier.py; src/character_segmenter.py; src/preprocessing.py.
- [S4] src/whole_line_reader.py; src/paddle_candidate.py; training/adapt_combined_paddle.py; model provenance files.
- [S5] docs/FORMAT_AWARE_UPDATE.md; src/format_decoder.py; src/validator.py; outputs/evaluation/format_aware_comparison.json.
- [S6] docs/READING_ORDER_FIX.md; docs/OWN_MODEL_100_REVIEWS.md; docs/CNN_CONSERVATIVE_UPDATE.md.
- [S7] docs/CNN_300_REVIEW_RESULTS.md; docs/CNN_SEGMENTATION_TRIALS.md; docs/FINAL_CNN_REVIEW.md.
- [S8] docs/COMBINED_DATASET_TRAINING.md; docs/NEW_DATASET_TRIAL.md; docs/SECOND_DATASET_TRIAL.md; outputs/evaluation/combined_training_summary.json.
- [S9] docs/PADDLE_PROJECTION_UPDATE.md; models/paddle_combined_v2/adaptation.json; training_report.json; development_evaluation.json.
- [S10] outputs/evaluation/combined_detector_before.json; combined_detector_after.json; combined_detector_full_epoch.json.
- [S11] outputs/evaluation/character_metrics.json; outputs/evaluation/combined_new_validation.json.
- [S12] docs/OCR_THRESHOLD_090.md; docs/CNN_THRESHOLD_070.md; outputs/evaluation/cnn_threshold_comparison.json; ocr_threshold_comparison.json.
- [S13] docs/DATASETS.md; docs/REAL_READER.md; docs/CHARACTER_REVIEW.md; docs/OCR_TEXT_REVIEW.md; src/plate_reader.py.

Official upstream projects

- [E1] [PaddleOCR](#)
- [E2] [RapidOCR / ONNX deployment](#)
- [E3] [Open Image Models / plate_detector](#)
- [E4] [Torchvision SSDLite320 MobileNetV3-Large](#)

Upstream references establish attribution and component identity. All accuracy figures in this report come from the local project records, not vendor benchmarks. Calculated percentages are rounded to one decimal place. Report prepared 4 October 2026.